

BioNix_Week_8_Wesley_ILP

Tuesday, 9 September 2025

17:41

Week 8 – ILP (WEHI BioNix)

What I did

- Built my first reproducible BioNix package: a **samtools wrapper** using symlinkJoin.
- Proved reproducibility by pinning nixpkgs to nixos-24.05 and getting identical store paths across rebuilds.
- Set up a repo locally and committed the wrapper + README; began sorting out remotes/branches.

Key code (and what it means)

```
{ pkgs ? import <nixpkgs> {} }:  
pkgs.symlinkJoin {  
  name = "samtools-wrapper-${pkgs.samtools.version}";  
  paths = [ pkgs.samtools ];  
}
```

- pkgs.symlinkJoin: creates a new output by **symlinking** binaries from pkgs.samtools into one \$out—no compile, no sources, just a clean wrapper.
- Benefit: tiny, fast, and perfect as a **reproducible baseline tool** inside BioNix.

How I built and verified

- Pinned build:
nix-build -I nixpkgs=https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz bionix-examples/samtools/samtools.nix → /nix/store/...-samtools-wrapper-1.19.2 (same path on repeated builds).
- Version check: ./result/bin/samtools --version printed **samtools 1.19.2** (using htlib 1.19.1).

Issues I hit (and why they happened)

- **Pasted Nix into bash** → “command not found” (because the shell tried to run Nix syntax). Fixed by saving to samtools.nix and building with nix-build.
- **Builder failed: variable \$src or \$srcs should point to the source** when using mkDerivation without disabling phases. Root cause: mkDerivation expects a source; for wrappers we must use dontUnpack/dontBuild or switch to symlinkJoin.
- **File not found** (samtools.nix): I invoked from ~ after moving the file. Fixed by running the command in the correct directory or using the full path.
- **Unpinned build drifted to samtools 1.22.1**: running nix-build samtools.nix (no -I nixpkgs=...) fetched a newer weekly nixpkgs, changing the output path/version. Lesson: always pin.
- **Git setup errors**:
 - “not a git repository” before git init.
 - “Author identity unknown” until I set user.name/user.email.
 - Push failures (“src refs spec main...”) because I was on master and/or pointing at the upstream lab repo without permissions; solution is to push to my **own fork** (or rename branch to main).

Evidence (CLI excerpts)

- Repeated pinned builds → identical store path: /nix/store/49l8...-samtools-wrapper-1.19.2.
- samtools --version output captured in the log.

Deliverables this week

- bionix-examples/samtools/samtools.nix (symlinkJoin wrapper).
- README.md with pin, build, verify, test steps. (Created and committed.)

Next week – plan

1. **Repo hygiene:** move to a dedicated ~/bionix repo and push to **my GitHub fork**; add upstream remote for PapenfussLab (PR workflow).
2. **Add a flake** to pin nixpkgs permanently (remove -I flag from docs).
3. **Scale to a small multi-tool pipeline** (recommend: fastp → seqtk → samtools) packaged as a BioNix workflow; include an end-to-end test.
4. Optional stretch: try a trickier tool (e.g. RepeatMasker) once multi-tool skeleton is in place.

Some Extra Note

- **Consistency & speed:** Helpers encode org conventions (pinning, phases, patching, RPATH fixes, test hooks). Using them reduces one-off mistakes and makes reviews predictable.
- **Clarity vs control:** Raw syntax teaches fundamentals and helps in tricky edge cases, but increases boilerplate and drift. Helpers improve readability/maintainability but can hide mechanisms I should learn.

What I need decided: When should interns **prefer helpers by default** (e.g., all package defs use callPackage, standard phases, org overlay), and when is it acceptable/expected to drop to raw expressions for one-off research artifacts or teaching?

- **symlinkJoin** – best when I just want to **re-export existing binaries** (or merge multiple package trees) without rebuilding. Super fast; fails if you need to patch or add runtime env.
- **stdenv.mkDerivation + dontUnpack = true; dontBuild = true;** – for **wrapper derivations** that create launch scripts, set env vars, patch shebangs, or arrange paths. Use when you must run an install phase.
- **buildEnv** – to **compose a “virtual package”** from many paths (like a curated toolset) with a unified \$out and de-duplicated closures; good for toolkit bundles and teaching environments.
- **wrapProgram / makeWrapper** – when a program needs **runtime env tweaks** (e.g., PATH, PYTHONPATH, LD_LIBRARY_PATH, data dirs). Often used inside the installPhase of a mkDerivation wrapper.

Pitfalls to avoid:

- mkDerivation without dontUnpack → source error;
- file name collisions in symlinkJoin/buildEnv;
- hidden runtime deps (use wrappers instead of telling users to export env by hand);
- licensing (e.g., RepeatMasker may not be redistributable → likely **documented install hook** rather than publishing binaries).

1) Raw .nix vs helpers (nixpkgs/BioNix)

Meaning: “Raw” = hand-writing derivations/functions; **helpers** = idioms that encode best-practice (pinning, phases, composition).

Raw (teach/edge cases):

```
# pkgs/default.nix
{ pkgs ? import <nixpkgs> {} }:
```

```
pkgs.stdenv.mkDerivation {
  pname = "hello-src";
  version = "1.0";
  src = ./.; # explicit sources
  buildPhase = "true"; # custom phases
  installPhase = "mkdir -p $out/bin; cp ./hello $out/bin/";
}
```

Helper style (preferred/production):

```
# pkgs/hello/default.nix -- a function, no pkgs import inside
{ lib, stdenv, makeWrapper, hello }:
stdenv.mkDerivation {
  pname = "hello-wrapped";
  version = hello.version;
  dontUnpack = true; dontBuild = true;
  nativeBuildInputs = [ makeWrapper ];
  installPhase = "
    mkdir -p $out/bin
    makeWrapper ${hello}/bin/hello $out/bin/hello-wrapped \
      --set HELLO_MODE "wehi"
  ";
}

# top-level
{ pkgs ? import <nixpkgs> {} }:
pkgs.callPackage ./pkgs/hello { }
```

Rule of thumb: write **functions** and use `callPackage`. Use helpers to reduce boilerplate and align with team conventions; drop to raw only for teaching or weird one-offs.

2) symlinkJoin vs mkDerivation vs buildEnv vs wrap Program

symlinkJoin — re-export existing outputs (no build):

```
{ pkgs ? import <nixpkgs> {} }:
pkgs.symlinkJoin {
  name = "bio-core";
  paths = [ pkgs.samtools pkgs.bedtools ];
}
```

Use when you just need a **bundle of binaries**. Fastest.

mkDerivation (wrapper) — you need install logic/env tweaks:

```
{ pkgs ? import <nixpkgs> {} }:
pkgs.stdenv.mkDerivation {
  pname="samtools-wrapper"; version=pkgs.samtools.version;
  dontUnpack=true; dontBuild=true;
  nativeBuildInputs=[ pkgs.makeWrapper ];
  installPhase = "
    mkdir -p $out/bin
    makeWrapper ${pkgs.samtools}/bin/samtools $out/bin/samtools \
      --prefix PATH : ${pkgs.htslib}/bin
  ";
}
```

!

Use when you must **set PATH, add data, patch shebangs**.

buildEnv — curate a **toolchain** with collision handling:

```
{ pkgs ? import <nixpkgs> {} }:  
pkgs.buildEnv {  
  name = "teaching-env";  
  paths = [ pkgs.fastp pkgs.seqtk pkgs.samtools ];  
  extraOutputsToInstall = [ "bin" ];  
  ignoreCollisions = true;  
}
```

wrapProgram (via makeWrapper) — the **mechanism** that edits runtime env; typically used *inside* mkDerivation (see above). Choose this when a binary needs **env vars** (e.g., REF_DATA, PYTHONPATH) to run correctly.

Heuristic:

- Only expose binaries? → **symlinkJoin**
- Need scripts/env/patching? → **mkDerivation + makeWrapper**
- Bundle many tools as one “package”? → **buildEnv**
- Just tweak a single binary’s env? → **wrapProgram** inside mkDerivation

3) Canonical debug flow (what to type & why)

Eval vs build:

```
# Nix eval errors (syntax/attrs)  
nix-instantiate --show-trace your.nix
```

Read the build log:

```
nix-build your.nix || true  
nix log /nix/store/<hash>.drv
```

See what Nix *thinks* it will do:

```
nix show-derivation /nix/store/<hash>.drv # phases, src, env, inputs
```

Common fix for wrapper derivations:

```
dontUnpack = true; dontBuild = true; # prevent "$src/$srcs" errors
```

Interactive repro of failing phase:

```
nix-shell -E 'with import <nixpkgs> {}; mkShell { buildInputs=[ stdenv makeWrapper ]; }'  
# then manually run the commands from installPhase/buildPhase
```

Turn up verbosity / traces:

```
NIX_DEBUG=1 nix-build --show-trace your.nix
```

Pin drift check (the one you hit):

```
# BAD (drifts): nix-build your.nix  
# GOOD (pinned):  
nix-build -I nixpkgs=https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz your.nix  
# or use a flake with inputs.nixpkgs.url pinned
```



Week 8

Q: Since everything in Nix is an expression, where's the right balance between raw `.nix` and higher-level helpers (`nixpkgs/BioNix` utils)?

- I want to know when to prefer bare syntax vs abstractions in our codebase.

Q: When should I use `symlinkJoin` vs `stdenv.mkDerivation` (with `dontUnpack/dontBuild`) vs `buildEnv` vs `wrapProgram`?

- I wrapped `samtools`; I want an advise for future wrappers/pipelines.

Q: What's the canonical debugging flow for builder failures? (e.g., `nix log`, `inspect .drv`, `nix repl`, enable traces)

- I want a systematic checklist for diagnosing "builder failed with exit code 1".